Relatório de Orquestração SAGA

Projeto	Vehicle Resale API
Disciplina	Fase 5 – Dados e Segurança da Informação
Curso	Pós-Graduação FIAP – Arquitetura de Software
Data	Março/2026

Sumário

1. Contexto do Fluxo de Venda
2. Tipo de SAGA Utilizada
3. Fluxo da Transação SAGA
4. Justificativa da Escolha
5. Implementação no Código
6. Conclusão

---

1. Contexto do Fluxo de Venda

O processo de compra de veículos envolve múltiplas etapas que alteram o estado de dois recursos distintos — o **veículo** e a **venda**. Essa sequência caracteriza uma **transação distribuída** em que uma falha em etapa posterior exige um mecanismo de **compensação**.

Etapas do processo de compra:

Etapa	Operação	Responsável
1	Comprador solicita a compra via `POST /api/sales`	API REST
2	Sistema valida CPF do comprador no cadastro	`SaleService`
3	Veículo é marcado como **SOLD**	`VehicleController`
4	Venda é criada com status **PENDING**	`SaleService`
5	Código de pagamento (UUID) é retornado ao comprador	Resposta HTTP
6	Comprador efetua o pagamento externamente	—
7	Status do pagamento é atualizado via webhook	`POST /api/webhook/payment`

Problema de consistência: se o pagamento for rejeitado na etapa 7, o veículo permanece como `SOLD` — bloqueado no estoque — mesmo sem venda efetivada. É necessário um mecanismo de **compensação** para reverter esse estado.

---

2. Tipo de SAGA Utilizada

Foi adotada a **SAGA de Compensação** (*backward recovery*), na qual cada passo da transação possui uma ação compensatória capaz de desfazer seu efeito em caso de falha.

Comparativo: SAGA de Compensação vs. SAGA Orquestrada

Critério	SAGA de Compensação (adotada)	SAGA Orquestrada
Coordenação	Implícita, dentro do serviço	Orquestrador central externo
Complexidade	Baixa — 2 recursos, 1 ponto de falha	Alta — adequada para muitos serviços
Ferramentas extras	Nenhuma	Motor de estados (ex.: AWS Step Functions)
Rastreabilidade	Via logs e status da venda	Via dashboard do orquestrador
Adequação ao projeto	Ideal para o cenário atual	Recomendada em caso de expansão

---

3. Fluxo da Transação SAGA

3.1 Caminho feliz — pagamento aprovado

1. `POST /api/sales` é recebido com os dados do comprador e do veículo.
2. O CPF do comprador é validado no cadastro de clientes.
3. O veículo é marcado como **SOLD** (`VehicleController.markAsSold()`).
4. A venda é persistida com status **PENDING** e um UUID de pagamento é gerado.
5. O comprador recebe o `paymentCode`.
6. O webhook `POST /api/webhook/payment` chega com `paid: true`.
7. A venda é atualizada para status **APPROVED**.

Resultado: veículo permanece SOLD, venda APPROVED.

3.2 Caminho de falha — pagamento rejeitado (com compensação)

1. Passos 1 a 5 idênticos ao caminho feliz.
2. O webhook `POST /api/webhook/payment` chega com `paid: false`.
3. A venda é atualizada para status **REJECTED**.
4. **Ação compensatória:** `VehicleController.markAsAvailable()` é chamado, revertendo o veículo para status **AVAILABLE**.

Resultado: veículo volta ao estoque disponível, venda permanece como REJECTED para fins de histórico e auditoria.

3.3 Estados do veículo no fluxo

Momento	Status do veículo
Antes da venda	AVAILABLE
Após `POST /api/sales`	SOLD
Após webhook `paid: true`	SOLD (definitivo)
Após webhook `paid: false`	AVAILABLE (compensado)

> **Nota:** A venda com `paymentStatus = REJECTED` é mantida intencionalmente. O registro garante rastreabilidade completa do ciclo de vida da transação.

---

4. Justificativa da Escolha

a) Simplicidade do domínio

A solução possui dois recursos transacionais (veículo e venda) e um único ponto de falha tratado (rejeição do pagamento). Uma SAGA orquestrada com motor de estados externo adicionaria complexidade desnecessária.

b) Orquestração implícita suficiente

O `SaleService` já conhece ambos os recursos. A compensação pode ser executada diretamente, sem a necessidade de um mediador externo.

c) Consistência eventual aceitável

O intervalo entre a criação da venda e o processamento do webhook é curto. O veículo fica temporariamente como `SOLD` até o webhook chegar, o que é aceitável para este domínio de negócio.

d) Auditabilidade preservada

A venda com `paymentStatus = REJECTED` é mantida no banco, garantindo rastreabilidade completa do ciclo de vida de cada transação.

Evolução futura recomendada

Se a solução evoluir para incluir serviços adicionais — gateway de pagamento externo, serviço de notificações, geração de documentos — recomenda-se migrar para uma **SAGA Orquestrada** com um motor de estados, como **AWS Step Functions** ou **Apache Camel Saga**.

---

5. Implementação no Código

5.1 Componentes envolvidos

Componente	Camada	Responsabilidade
`SaleService.create()`	Domínio	Chama `markAsSold()` e persiste a venda com PENDING
`SaleService.updatePaymentStatus()`	Domínio	Atualiza para APPROVED ou REJECTED; aciona compensação se `paid == false`
`VehicleController.markAsSold()`	Aplicação	Atualiza o veículo para status SOLD
`VehicleController.markAsAvailable()`	Aplicação	Ação compensatória: reverte o veículo para AVAILABLE

5.2 Trecho relevante — compensação em `SaleService`

```
// SaleService.java — método updatePaymentStatus()
if (Boolean.FALSE.equals(paid)) {
    sale.paymentStatus = PaymentStatus.REJECTED;
    // Compensação SAGA: libera o veículo de volta ao estoque
    vehicleController.markAsAvailable(sale.vehicle.id);
} else {
    sale.paymentStatus = PaymentStatus.APPROVED;
}
```

5.3 Cobertura por testes

Arquivo	Caminho	Cenário coberto
`SaleServiceTest.java`	`src/test/java/ … /domain/service/`	Verifica que `markAsAvailable()` é invocado quando `paid = false`
`SaleService.java`	`src/main/java/ … /domain/service/`	Implementação da compensação
`VehicleController.java`	`src/main/java/ … /application/controller/`	Método `markAsAvailable()`

6. Conclusão

O padrão **SAGA de Compensação (backward recovery)** foi adotado por ser adequado ao porte e à complexidade da solução atual.

A compensação garante que um veículo nunca fique bloqueado com status `SOLD` após um pagamento rejeitado, mantendo a **consistência do inventário** e a **auditabilidade** de cada transação.

Resumo da solução:

Aspecto	Valor
Tipo de SAGA	Compensação — backward recovery
Orquestrador	Implícito (`SaleService` + `VehicleController`)
Ponto de falha tratado	Pagamento rejeitado (`paid: false`)
Ação compensatória	`VehicleController.markAsAvailable()`
Consistência	Eventual — aceitável para o domínio
Cobertura por teste	Sim — `SaleServiceTest`

Documento elaborado para a Fase 5 – Dados e Segurança da Informação – Pós-Graduação FIAP – Arquitetura de Software – Março/2026.